Abstract

HashLab is a Python and Streamlit application that demonstrates hashing through separate chaining, linear probing and double hashing. Users can insert, search and delete integer keys, inspect the table, and follow the most recent operation. A library mode uses book IDs to retrieve titles and shelf locations through the same custom hash-table engine.

The project studies how load factor and input distribution affect collision handling. Identical random and deliberately colliding datasets are used to compare insertion and search costs. The measurements count key comparisons and array visits rather than machine-dependent execution time. Results show that long chains and probe clusters increase work, while double hashing spreads the selected colliding keys more effectively. Both probing methods can inspect the entire table during an unsuccessful search at full capacity.

Thirteen automated tests passed, covering table operations, book records, duplicate IDs, deletion, full tables and interface workflows. The system is a demonstration application with session-based records; it is not a permanent library catalogue.

Contents

1 Introduction and Requirements 2

2 Hashing and Collision Resolution 3

3 System Design and Workflow 4

4 Algorithm Implementation 5

5 Library Book Lookup 6

6 Complexity and Implementation Details 7

7 Experimental Results 8

8 Testing and Discussion 9

9 Conclusion and References 10

1 Introduction and Requirements

1.1 Background and problem statement

Hashing provides a way to locate a record from its key without scanning every record. A hash function converts the key into an array index. Since the number of possible keys is larger than the number of array positions, different keys can reach the same index. The table must resolve this collision without losing or confusing the records.

The project makes this behaviour visible and measurable. It shows where an operation begins, which positions it visits and how many stored keys it compares. The library example gives these operations a practical purpose. Book IDs 12 and 23 both start at index 1 in a table of size 11, but a search must still return the correct title and shelf for the requested ID.

1.2 Objectives

- Implement insertion, search and deletion using three collision strategies.
- Display the table and the steps of the most recent operation.
- Compare strategies using the same keys, order, capacity and search queries.
- Demonstrate record lookup through a small library example.

1.3 Scope and requirements

The Simulator supports integer keys and library books. Users can choose a prime capacity, load five colliding entries, clear the table and inspect a trace. The comparison tab generates random or same-bucket input and exports measurements as CSV. Borrowing, fines, accounts, editing records and permanent storage are outside the current scope.

The application requires Python 3.10 or newer, Streamlit, pandas and Altair. The version ranges are recorded in requirements.txt. It runs locally in a browser and needs no database, API key or external service after dependency installation.

Library IDs must be positive integers. Adding a book requires a non-blank title and shelf. Spaces at the start and end are removed; the fields are limited to 120 and 40 characters respectively. A repeated ID keeps its existing record. Changing mode, strategy or capacity clears the simulator, as does loading a replacement sample.

2 Hashing and Collision Resolution

2.1 Hash function and load factor

HashLab uses the division method, where m is the number of slots or buckets:

$$h(k) = k \bmod m$$

The index lies between 0 and m - 1. It is a starting location, not a unique identity. For example, 12, 23 and 34 all have remainder 1 when divided by 11.

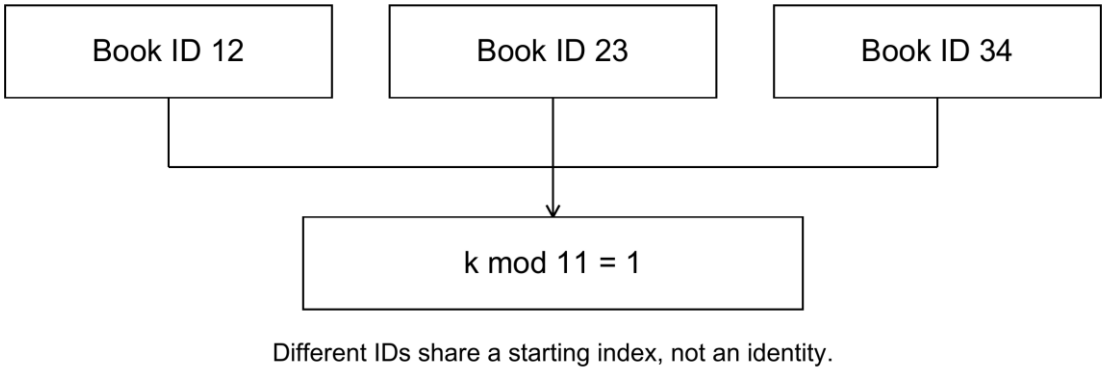


Figure 1. Different book IDs sharing an initial index

If n keys are stored, the load factor is:

$$\alpha = \frac{n}{m}$$

Five entries in 11 buckets give a load factor of about 0.45. Open addressing has one live key per slot, so its load cannot exceed 1. Chaining can exceed 1 because each bucket holds a linked list.

2.2 Implemented strategies

Table 1. Collision strategies in HashLab

Strategy	Storage	Next candidate
Separate chaining	Linked nodes in a bucket	Next node
Linear probing	One key per array slot	Next slot
Double hashing	One key per array slot	Key-dependent step

Linear probing uses step 1. Double hashing uses 1 + (k mod (m - 1)). All capacities are prime, so every possible nonzero step is relatively prime to m and can visit all slots. This complete-cycle condition is important for correct probing [1]. Standard algorithm analysis describes good expected lookup cost under suitable hashing and load assumptions [1].

3 System Design and Workflow

The system has three main modules. app.py handles the Streamlit controls, validation and session state. hashing.py implements the custom tables and returns results and traces. experiments.py calls the same engine with controlled datasets. styles.css handles display styling.

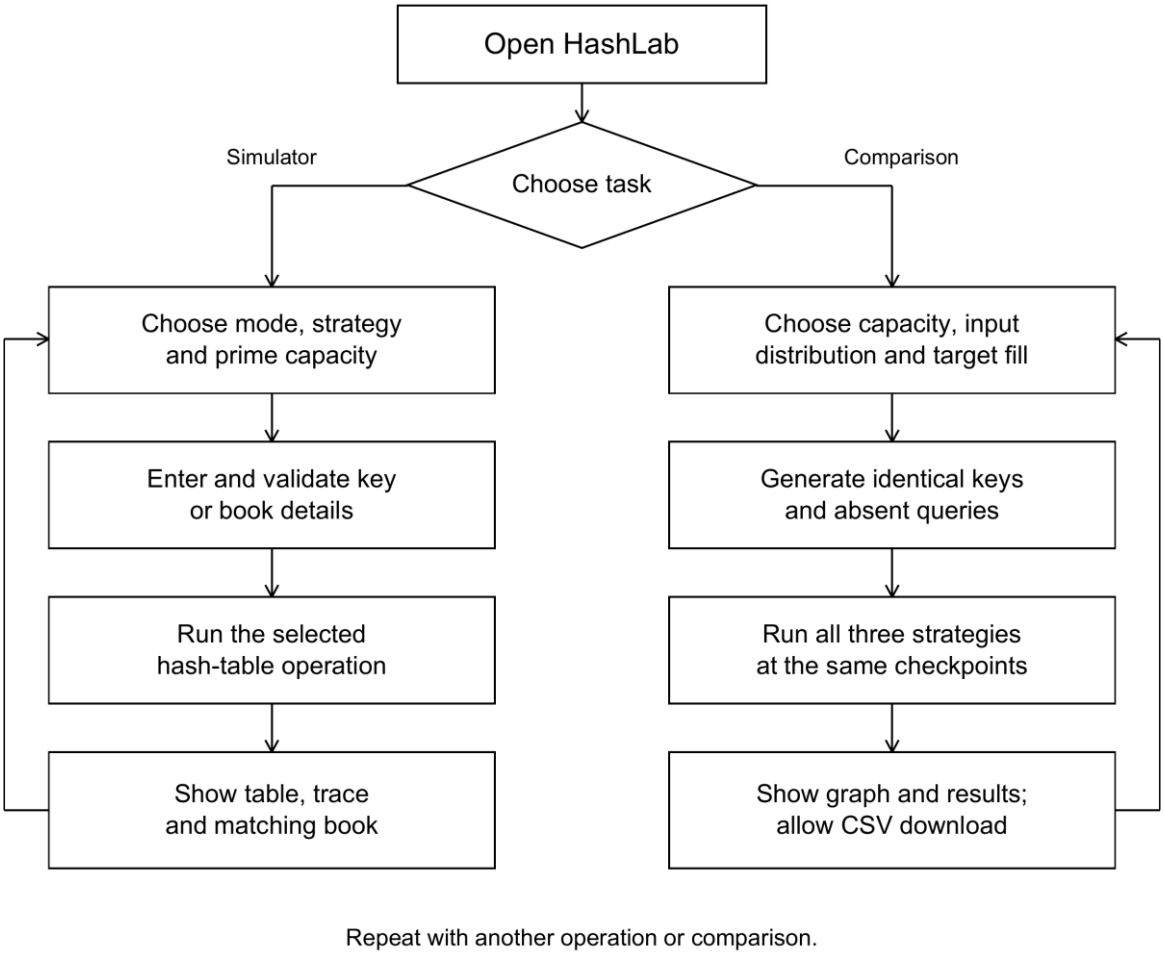


Figure 2. Complete workflow of the simulator and comparison module

The table, previous snapshot, latest result, collision total and trace position are held in session state [2]. They last for the current session. Comparison results are stored separately from the simulator. A Result contains the status, visits, comparisons, insertion collisions, trace steps and, when available, the matching book.

The operation selector is outside the input form so that it can show the correct fields immediately. Submitted values are processed before the table and results are drawn. Forms group values into one submission [2], and no extra forced rerun is needed.

4 Algorithm Implementation

4.1 Separate chaining

Each bucket stores the head of a linked list. A node holds a key, an optional Book record and the next reference. Insertion scans for a duplicate before appending a node. Search stops at a full-key match or the end of the list. Deletion unlinks only the matching node, including the head when necessary.

4.2 Linear probing and double hashing

Both probing methods use the following sequence, with i starting at zero:

$$p(i,k) = (h(k) + i * step(k)) \, mod \, m$$

```
PROBE_INSERT(key, book)
    first_deleted = NONE
    for index in at most m probe positions:
        if slot is DELETED:
            remember first deleted index; continue
        if slot is EMPTY:
            store in first_deleted or current index; return
        if slot contains key: return ALREADY_PRESENT
        count the unequal live key as a collision
    if first_deleted exists: store there
    otherwise: return TABLE_FULL
```

A deleted slot is remembered, but does not end the duplicate check. Search also continues past deleted markers. A never-used empty slot ends a failed search. Every probe loop is bounded by m positions, so a full table cannot cause an infinite loop.

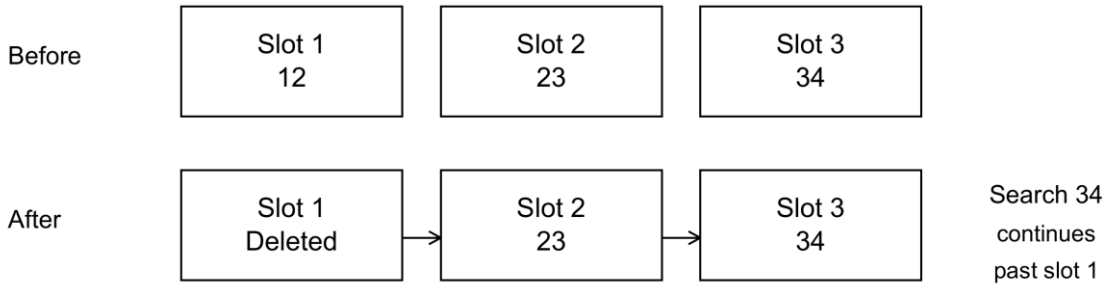


Figure 3. Deletion preserves the path to later keys

After removing 12, a search for 34 must pass the marker at slot 1 and compare the later keys. Treating slot 1 as empty would stop the search too early. Deletion also clears the book record at that slot. Matching always checks the full ID rather than only its hash index.

5 Library Book Lookup

5.1 Records and operations

Library mode supports Add book, Find book and Remove book. The ID is the integer key; the record contains a title and shelf location. Chaining stores the record in its linked node. Probing stores the record at the same array index as the key. Book retrieval therefore uses the custom table, not a separate Python dictionary.

Table 2. Collision sample with capacity 11

Book ID	Title	Shelf
12	Algorithms	A2
23	Python Basics	B3
34	Data Structures	C4
45	Computer Networks	D5
56	Database Systems	E6

All five sample IDs start at index 1. Chaining stores them in one list. Linear probing stores them in slots 1 to 5. Double hashing places them at indices 1, 5, 6, 7 and 8 respectively. These are demonstration records, not a real library dataset.

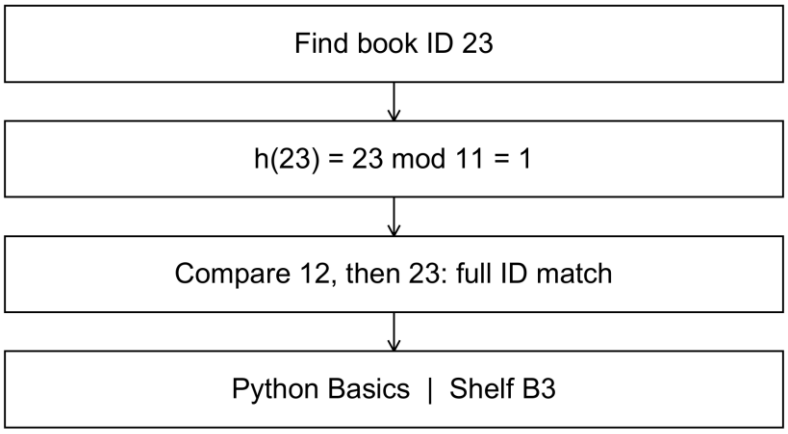


Figure 4. Finding the book with ID 23 using separate chaining

5.2 Duplicate and deletion behaviour

Finding ID 23 returns Python Basics on shelf B3. With the sample loaded, this takes two comparisons in each strategy. Chaining visits one bucket; either probing method visits two slots. After removing ID 12, ID 23 remains searchable. Chaining needs one comparison; probing still visits two slots but makes only one live-key comparison because the first slot is deleted.

Adding an existing ID reports Already present and keeps its title and shelf unchanged. Removing an absent ID reports Not found and does not change the entry count. Records are session-only, and switching the table configuration starts a new table.

6 Complexity and Implementation Details

6.1 Correctness and termination

In chaining, every key remains in the bucket selected by its initial hash. Insertion adds a node only after checking for an equal key; deletion changes links without disconnecting the remaining records. In probing, insertion and search use the same sequence. A valid step visits all slots, and deleted markers preserve the sequence after removal.

A never-used empty position safely ends search because insertion would have stopped at that position before placing the same key farther along. A full cycle ends after m visits. Equal starting indices are never enough to identify a book: only a full-key match returns its associated record.

6.2 Time and space requirements

Table 3. Complexity of the implemented tables

Measure	**Separate chaining**	**Open addressing**
Expected search	O(1 + alpha), under uniform hashing	Expected O(1) with suitable hashing and controlled load
Worst operation	O(n) node checks	O(m) slot visits
Table storage	O(m + n), plus record data	O(m), plus record data
Longest trace	O(n)	O(m)

Here n is the number of keys and m is capacity. If n and m are of the same order, a full probe cycle is linear in the collection size. Chaining may have n greater than m. This implementation scans a chain before appending, so insertion into a long chain is not unconditionally constant time.

6.3 Counting rules and display cost

A bucket or slot visit counts an inspected array position. A key comparison counts equality testing against a live stored key. Chaining visits one array bucket and then compares linked nodes. An insertion collision counts an unequal live key encountered during an insertion attempt. Empty positions and deleted markers cost visits but no live-key comparison.

The interface displays the state before an operation at earlier trace steps and the completed state at the final step. It does not animate every intermediate mutation. Snapshot creation and drawing have their own cost and are not included in the engine counters. Experiments disable trace collection to keep the comparison focused on table operations.

7 Experimental Results

7.1 Method

The reported runs use capacity 101 and seed 42. Each strategy receives identical unique keys, order and queries. At 90% target fill, 90 keys are inserted, giving actual load 90/101 = 0.8911. Random keys and absent queries come from disjoint integer ranges using random.sample. Colliding keys follow 3 + 101i; absent queries continue beyond the inserted range.

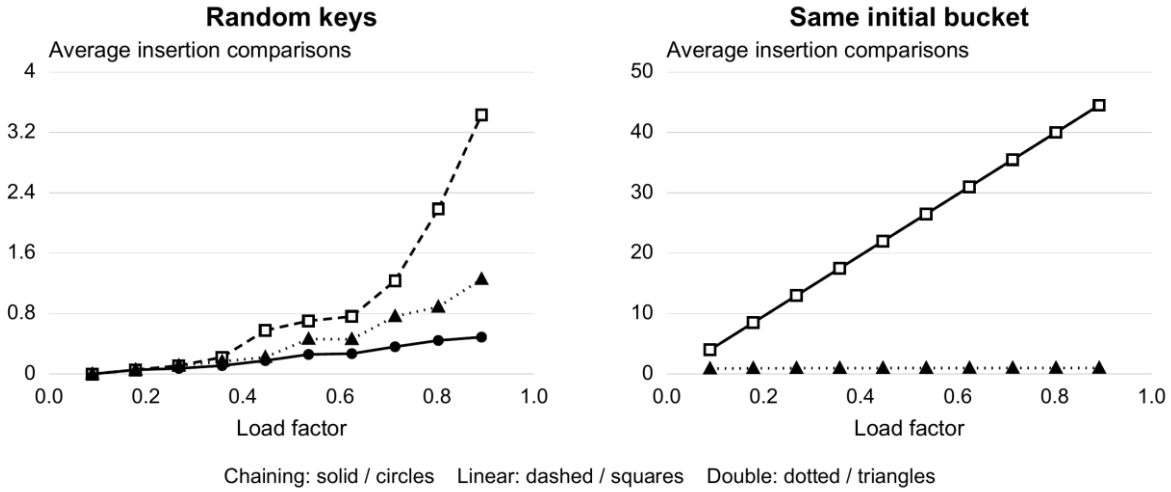


Figure 5. Average insertion comparisons as the load factor increases

Up to ten checkpoints are measured. Insertion averages cover all keys inserted so far. Successful searches cover every present key; unsuccessful searches use the same fixed sample of 100 absent keys. Table 4 reports average live-key comparisons at the final checkpoint.

Table 4. Average comparisons at 90 keys and capacity 101

Input	Strategy	Insert	Found	Absent
Random	Separate chaining	0.49	1.49	1.10
Random	Linear probing	3.43	4.43	18.29
Random	Double hashing	1.26	2.26	8.01
Colliding	Separate chaining	44.50	45.50	90.00
Colliding	Linear probing	44.50	45.50	90.00
Colliding	Double hashing	0.99	1.99	7.29

With colliding input, chaining and linear probing each make 4,005 insertion comparisons, or 44.50 per key. Double hashing makes 89, averaging 0.99. Its secondary steps spread these particular keys. Keys sharing both hash components can still follow the same probe path, so this is not a worst-case guarantee.

8 Testing and Discussion

8.1 Automated validation

Thirteen unittest and Streamlit AppTest tests passed: eight cover the engine and experiments, and five cover interface workflows. The main checks are:

- Colliding keys remain searchable, and duplicates do not increase the count.
- Deletion preserves other records; full probing stops and deleted slots can be reused.
- Double-hash sequences cover every slot for 200 tested keys at five prime capacities.
- Eight hundred random operations per strategy agree with a reference set.
- The same experiment settings reproduce the same measurements.
- Book details survive collisions, duplicates, deletion and slot reuse.
- The interface validates input, loads samples, runs comparisons and resets correctly.

The reference set is used only as a test oracle. It does not replace the custom table in the application. The library tests confirm that a full-table insertion failure does not create a hidden record and that an existing ID retains its original title.

8.2 Interpretation of the results

The random workload produces short average chains, while probing becomes more expensive as occupancy rises. In the deliberately colliding workload, chaining forms one long list and linear probing forms one cluster. Their equal comparison counts do not imply equal execution time or memory behaviour.

A separate random run stores 101 keys at capacity 101. Both probing strategies then make 101 comparisons for every absent query because no empty stopping slot remains. Chaining averages 1.10 comparisons for the same absent-query sample. Its capacity describes the number of buckets, not a fixed limit on nodes.

The experiments show that both occupancy and key distribution matter. They use one seed and constructed datasets, and do not measure elapsed time, memory consumption or usability with library staff. Passing tests supports correctness in the tested cases; it does not establish production performance.

8.3 Reproducing the application and tests

```
python -m pip install -r requirements.txt
python -m streamlit run app.py
python -m unittest discover -s tests -v
```

9 Conclusion and References

9.1 Conclusion

HashLab connects collision-resolution algorithms with visible table operations and a small library lookup. It preserves the correct book record when different IDs share an index. Chaining uses linked nodes; probing uses bounded array sequences and deleted markers. The same engine supplies the simulator and comparison module.

The measurements show increasing work in long chains and occupied probe regions. Double hashing spreads the chosen colliding sequence more effectively, but it still scans the full table for an absent key at full capacity. Good expected performance therefore depends on suitable hashing, the input distribution and controlled load.

9.2 Limitations and future work

Records last only for the current session. The application has numeric-ID lookup but no title search, editing, lending or permanent catalogue. The table does not resize or clean accumulated deleted markers. Useful extensions would be a small persistence layer, optional rehashing, and comparisons across more seeds and keys that share both hash components. These should preserve the project's focus on explaining the data structure.

References

[1] Cormen et al. Introduction to Algorithms.

[2] Streamlit documentation. https://docs.streamlit.io/

Figures and measurements were generated from the HashLab implementation.